Hybrid Scheduling & Dual Queue Scheduling

KEVIN & SCOTT

S. N. M. Shah, A. K. B. Mahmood and A. Oxley, 2009 IEE

MOTIVATION & PROBLEM STATEMENT

WHY CPU SCHEDULING

The CPU is the most important resource in an OS with scheduling being a fundamental function of a CPU. Efficient scheduling is essential to system throughput and user experience.

ROOM TO IMPROVE

Classic algorithms we previously learned trade off either starvation, fairness, and/or response time. No single algorithm perfectly handles each metric simultaneously.

RESEARCH GOAL

Create new algorithms that optimize waiting time, turnaround time, and response time

CLASSIC ALGORITHM

FCFS

Key Idea:

- First Come, First Served: processes execute in order of arrival

Advantages:

- Minimal overhead (less context switches)
- Easy to implement, best used for batch jobs with similar size

Disadvantages:

- Suffers from the *Convoy Effect*
- Worst TAT of all algorithms
- Non-preemptive, no fairness for late jobs

CLASSIC ALGORITHM

SJF

Key Idea:

- Shortest Job First: jobs queue by CPU burst length, shortest first

Advantages:

- Optimal for minimizing TAT and WT compared to other non-preemptive algorithms

Disadvantages:

- Starvation Risk
- Requires knowing burst lengths
- Non-preemptive, mid RESP

CLASSIC ALGORITHM

STCF

Key Idea:

- Shortest Time-to-Completion: preemptive version of SJF; new arrivals preempt if they have shorter remaining times

Advantages:

- Best TAT of all algorithms with late arrivals
- Strong WT

Disadvantages:

- Starvation Risk
- Requires knowing burst lengths
- High overhead if preemptions are frequent, mid RESP

CLASSIC ALGORITHM

RR

Key Idea:

- Round Robin: FIFO queue with quantum; preempts on expiry, moves to tail

Advantages:

- Fair CPU sharing, no starvation
- Best RESP

Disadvantages:

- Poor TAT
- Quantum size affects performance
  - Too large = similar to FCFS, Too small = context switch overhead

CLASSIC ALGORITHM

MLFQ

- Add your first bullet point here
- Add your second bullet point here
- Add your third bullet point here

NEW ALGORITHM

HYBRID (H)

Key Idea:

- Combines RR + SJF, runs processes in a SJF queue with a quantum
- Periodically forces the *largest* job a turn, then re-sorts the queue back to SJF

Advantages:

- Best WT of all algorithms
- Strong TAT
- No starvation

Disadvantages:

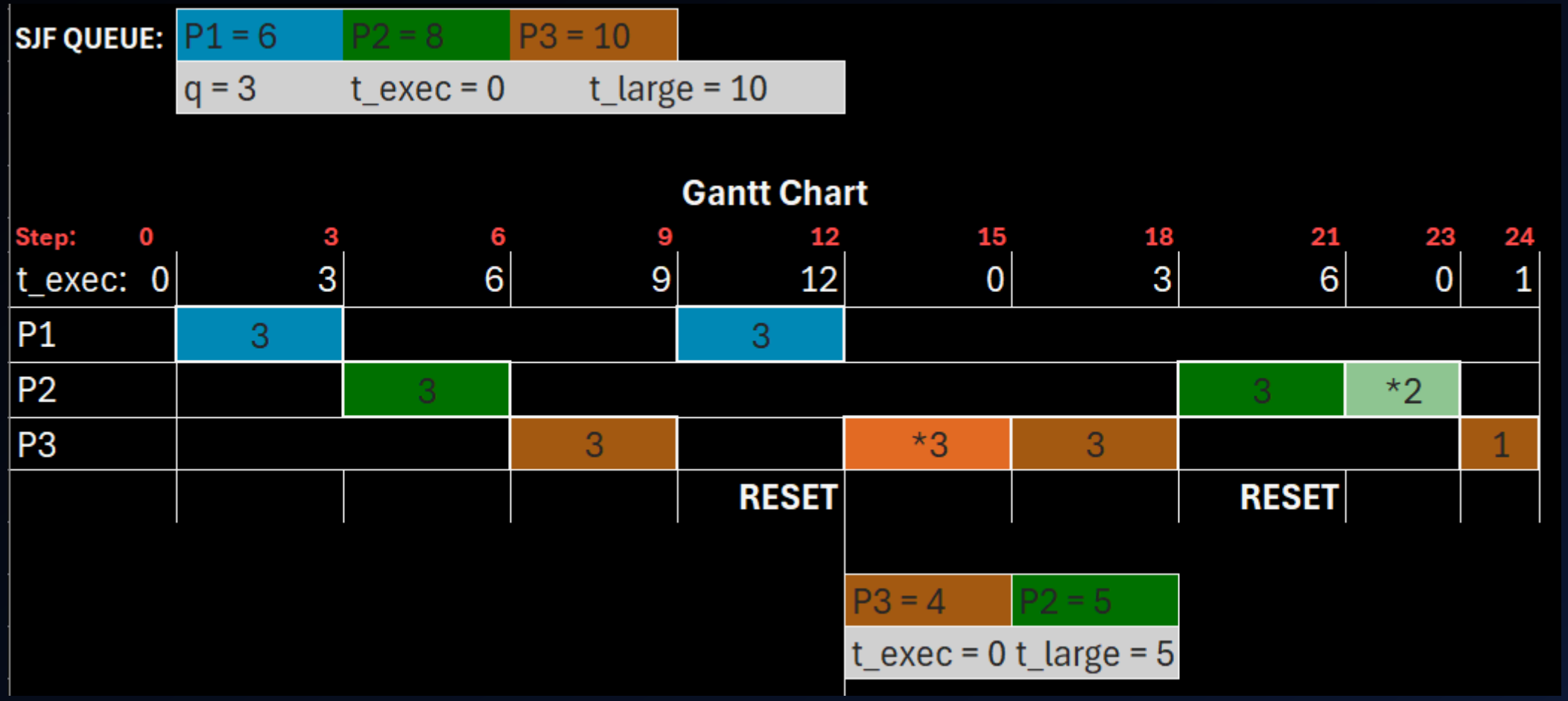
- More complex than classic algorithms
- Jobs need to arrive at the same time, needs an oracle
- Sorting overhead at each reset cycle

H – A Deeper Look

- **The Two Key Variables:**
  - t_{large} = burst length of the longest process currently in the queue
  - t_{exec} = CPU time elapsed since last reset, starts at 0

- **Execution Flow**
  1. Start the process at the head of the queue
  2. After each quantum expires, update t_{exec} $(t_{exec} = t_{exec} + q)$
  3. Compare t_{exec} & t_{large}:
     - If $t_{exec} < t_{large}$ → normal operation, current process goes to the back of the queue. Start the next process in queue
     - If $t_{exec} \geq t_{large}$ → triggers reset, largest job immediately gets a turn. Queue is resorted to SJF. t_{exec} resets to 0. t_{large} updates to current largest process. Start next process in queue

H – A Simple Example



*made by us in excel

NEW ALGORITHM

DUAL QUEUE (DQ)

Key Idea:

- Expands on H but with 2 queues (FIFO waiting queue + SJF execution queue)
- New arrivals are batched into the waiting queue and get flushed into the execution queue on reset

Advantages:

- Best RT of all algorithms
- Lower overhead than H
- No starvation

Disadvantages:

- Even more complex logic than other algorithms
- Sorting overhead at each reset cycle

EXPERIMENT SETUP & METRICS

- Turnaround Time (TAT) = $T_{completion} - T_{arrival}$
- Response Time (RESP) = $T_{first\ run} - T_{arrival}$
- Waiting Time = Time spent waiting in the waiting queue

EXPERIMENT RESULTS

CONNECTION TO CORE OS CONCEPTS

CRITICAL REFLECTION

REFERENCES